

Stable Hypergraph Matching in Unimodular Hypergraphs

Péter Biró ✉ 

HUN-REN Centre for Economic and Regional Studies, Budapest, Hungary

Gergely Csáji ✉ 

HUN-REN Centre for Economic and Regional Studies, Budapest, Hungary
Eötvös Loránd University, Budapest, Hungary

Ildikó Schlotter ✉ 

HUN-REN Centre for Economic and Regional Studies, Budapest, Hungary

Abstract

We study the NP-hard STABLE HYPERGRAPH MATCHING (SHM) problem and its generalization allowing capacities, the STABLE HYPERGRAPH b -MATCHING (SH b M) problem, and investigate their computational properties under various structural constraints. Our study is motivated by the fact that Scarf's Lemma [19] together with a result of Lovász [15] guarantees the existence of a stable matching whenever the underlying hypergraph is normal. Furthermore, if the hypergraph is unimodular (i.e., its incidence matrix is totally unimodular), then even a stable b -matching is guaranteed to exist. However, no polynomial-time algorithm is known for finding a stable matching or b -matching in unimodular hypergraphs.

We identify subclasses of unimodular hypergraphs where SHM and SH b M are tractable such as laminar hypergraphs or so-called subpath hypergraphs with bounded-size hyperedges; for the latter case, even a maximum-weight stable b -matching can be found efficiently. We complement our algorithms by showing that optimizing over stable matchings is NP-hard even in laminar hypergraphs. As a practically important special case of SH b M for unimodular hypergraphs, we investigate a tripartite stable matching problem with students, schools, and companies as agents, called the UNIVERSITY DUAL ADMISSION problem, which models real-world scenarios in higher education admissions.

Finally, we examine a superclass of subpath hypergraphs that are normal but not necessarily unimodular, namely subtree hypergraphs where hyperedges correspond to subtrees of a tree. We establish that for such hypergraphs, stable matchings can be found in polynomial time but, in the setting with capacities, finding a stable b -matching is NP-hard.

2012 ACM Subject Classification Theory of computation → Design and analysis of algorithms; Theory of computation → Algorithmic mechanism design; Mathematics of computing → Hypergraphs

Keywords and phrases stable hypergraph matching, Scarf's Lemma, unimodular hypergraphs, university dual admission

Digital Object Identifier 10.4230/LIPIcs.ICALP.2025.31

Category Track A: Algorithms, Complexity and Games

Related Version *Full Version:* [arXiv:2502.08827](https://arxiv.org/abs/2502.08827) cs.GT [4]

Funding Supported by the Hungarian Academy of Sciences under Momentum grant no. LP2021-2. Péter Biró: Supported by the Hungarian Scientific Research Fund (OTKA grant no. K143858).

Gergely Csáji: Supported by the Hungarian Scientific Research Fund (OTKA grant no. K143858) and by the Ministry of Culture and Innovation of Hungary from the National Research, Development and Innovation fund, financed under the KDP-2023 funding scheme (grant no. C2258525).

Ildikó Schlotter: Supported by the Hungarian Academy of Sciences under its János Bolyai Research Scholarship.

Acknowledgements We are grateful for the reviewers of ICALP 2025 for their valuable comments.



© Péter Biró, Gergely Csáji, and Ildikó Schlotter;
licensed under Creative Commons License CC-BY 4.0

52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025).

Editors: Keren Censor-Hillel, Fabrizio Grandoni, Joël Ouaknine, and Gabriele Puppis; Article No. 31; pp. 31:1–31:20

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany



1 Introduction

Stable matchings are fundamental in economics, combinatorial optimization, and mechanism design, playing a crucial role in applications such as college admissions, job markets, and organ exchange programs. Since the seminal work of Gale and Shapley [12], stable matchings have been extensively studied in two-sided markets where agents form pairwise relationships. However, many real-world scenarios require interactions beyond pairwise relationships, leading naturally to hypergraph-based formulations.

The focus of our study is the **STABLE HYPERGRAPH b -MATCHING** (SH b M) problem, where we are given a hypergraph whose vertices represent agents, with each agent v having preferences over the incident hyperedges and, additionally, a capacity value $b(v)$. The task is to find a stable b -matching, i.e., a set M of hyperedges where each agent v is adjacent to at most $b(v)$ hyperedges in M , and no hyperedge e outside M is “desirable” for all agents in e ; see Section 2 for the precise definition of stability.

In the context of hypergraph matchings, stability is often difficult to achieve, as stable solutions may not always exist. On the positive side, Scarf’s Lemma [19] guarantees the existence of a fractional stable solution in the SH b M problem. Furthermore, a famous result of Lovász [15] in combination with Scarf’s Lemma ensures that in an instance of **STABLE HYPERGRAPH MATCHING** (SHM) – the restriction of SH b M without capacities – with a *normal hypergraph*, a stable matching always exists. An even stronger result follows for *unimodular hypergraphs*, where the incidence matrix is totally unimodular: Scarf’s Lemma guarantees the existence of a stable b -matching. However, these results only provide existential guarantees, leaving computational aspects largely unexplored.

Motivated by these structural insights, we investigate the computational complexity of finding stable (b -)matchings in different hypergraph classes. Furthermore, we consider not only the problem of finding *any* stable (b -)matching but also of computing maximum-weight stable solutions.

1.1 Our Contribution

We begin with a class inspired by a real-world problem: the University Dual Admission (UDA) problem which arises in higher education admissions where students apply to universities and internship programs simultaneously [10]. Although we show that a stable solution always exists by reducing UDA to SH b M in a unimodular hypergraph, our proof does not directly lead to an efficient algorithm. though we are unable to settle the complexity of finding a stable solution in a general instance of UDA, we provide an algorithm that finds a maximum-weight stable solution and runs in XP time parameterized by the number of universities. To complement this, we prove that computing a maximum-size stable matching is NP-hard even without capacities, although we show that a stable matching can be found efficiently in this case. Additionally, we introduce a relaxation of stability, termed *half-stability*, and design a polynomial-time algorithm that always finds a half-stable matching.

Next, we investigate *laminar hypergraphs*, a class where hyperedges follow a hierarchical inclusion property. Leveraging their structure, we design a polynomial-time algorithm that finds a stable b -matching whenever the input hypergraph is laminar. By contrast, we also establish that, even in the setting without capacities, finding a maximum-weight stable matching (or even a stable matching containing a fixed edge) is NP-hard.

We extend our study to a superclass of laminar hypergraphs, *subpath hypergraphs*, where hyperedges can be represented as subpaths of a path. Subpath hypergraphs naturally model scenarios where relationships or interactions are constrained along a linear structure, such as transportation networks or supply chains. We develop an XP algorithm parameterized by the largest hyperedge size for finding a maximum-weight stable b -matching in such hypergraphs.

■ **Table 1** Computational complexity of finding an arbitrary or a maximum-weight stable (b)-matching in different hypergraph classes. **NPh**, **P**, **XP**, and **W[1]h** refers to **NP**-hardness, polynomial-time solvability, solvability in **XP**, and **W[1]**-hardness (the latter two with the given parameter), respectively. Parameter ℓ_{max} is the size of the largest hyperedge and n_U is the number of universities. Recall the hierarchy of hypergraphs which we may write informally as $\{\text{laminar}\} \subseteq \{\text{subpath}\} \subseteq \{\text{subtree}\}$.

	Matching		b-Matching	
	Stable	Max-Weight Stable	Stable	Max-Weight Stable
UDA	P [R14]	NPh , XP (n_U)[T10,T11]	?, XP (n_U)[T11]	NPh , XP (n_U)[T10,T11]
Laminar	P [7]	NPh , XP (ℓ_{max}) [T17,T18]	P [T15]	NPh , XP (ℓ_{max}) [T17,T18]
Subpath	P [7]	NPh , XP (ℓ_{max}) [T17,T18]	?, XP (ℓ_{max}) [T18]	W[1]h , XP (ℓ_{max}) [T24,T18]
Subtree	P [T25]	NPh [T17]	NPh [T26]	NPh [T17]

Finally, we consider a superclass of subpath hypergraphs, *subtree hypergraphs*, where hyperedges correspond to subtrees of an underlying tree. While these hypergraphs are not necessarily unimodular, they are still normal, which ensures the existence of a stable matching by the results of Lovász [15] and Scarf [19]. For the matching case without capacities, we provide a polynomial-time algorithm, which contrasts our result that computing a stable b -matching for general capacities is **NP**-hard.

Possible applications that motivate the study of **SHbM** for various hypergraph classes include problems related to budgeting in transportation networks, project selection management in companies, or coalition formation in politics; see the full version of our paper [4].

Our findings, summarized in Table 1, contribute to the understanding of stable matchings in structured hypergraphs, bridging theoretical guarantees with computational feasibility.

1.2 Related Work

In their seminal paper, Gale and Shapley [12] provided a model and an algorithmic solution for college admission problems. Their solution concept is called *stability*, which means that an application is fairly rejected by a college if the college filled its quota with better applicants. Gale and Shapley showed that their so-called deferred-acceptance (DA) algorithm always finds a stable solution that is optimal for the students if they are proposing in the algorithm.

The work of Gale and Shapley [12] inspired extensive research in mathematics, computer science, operations research, as well as in economics and game theory. For an overview, we recommend the reader to consult the book of Manlove [16] in computer science, the books of Roth and Sotomayor [18] and Haeringer [13] in economics. Besides the theoretical research, the DA algorithm has also been used in many important applications, firstly in the US resident allocation scheme (called NRMP) since 1952 [17] and also in several nationwide college admission and school choice programs.

One of our motivating problems originates in the Hungarian university scheme where it is possible to apply for a so-called *dual program* that consist of a normal university major together with an internship at a company. For a student to get accepted to such a dual program, both the university and the company have to grant admission, and these parties can have different rankings. Note that this feature is not integrated in the current admission system, as the hiring decisions of the companies are not shared with the central coordinators when conducting the matching algorithm, which causes serious coordination problems, see a detailed description of the problem by Fleiner et al. [10].

Rather independently from the topic of matching under preferences, Scarf [19] showed that so-called balanced NTU-games (a family of cooperative games with non-transferable utility) have non-empty core by providing a finite algorithm that computes a core solution in such games. Instead of introducing these game-theoretical notions, we only interpret and use Scarf's Lemma in the context of stable matchings and b -matchings in hypergraphs.

The STABLE HYPERGRAPH MATCHING problem, introduced by Aharoni and Fleiner [1], is essentially equivalent to the problem of finding a core solution in simple NTU-games, also known as *core stability in hedonic coalition formation games* [3, 21, 2]; in the hypergraph formulation of these problems, vertices represent agents and hyperedges represent the possible coalitions. The setting where agents have capacities, giving rise to the SHbM problem, was introduced by Biró and Fleiner [5] who proposed a general framework and proved the existence of fractional stable solutions using Scarf's Lemma for b -matching problems. They also highlighted that the matching polytope has the integer property for normal hypergraphs, therefore the solution determined by Scarf's algorithm for the SHM problem is always integer and thus corresponds to a stable matching.

However, the running time of computing a fractional solution by Scarf's algorithm is not polynomial in general. The PPAD-hardness of computing a fractional stable matching was proved by Kintali et al. [14] and was linked with several other PPAD-hard combinatorial problems. Recently Csáji extended these PPAD-hardness results for the fractional stable matching problem for three-partite hypergraphs and for the stable allocation with couples problem [8]. On the positive side, for certain stable matching problems it was shown that Scarf's algorithm terminates in polynomial time for some appropriately defined pivot rule. Very recently Faenza et al. [9] showed this property for the classical stable marriage problem, and Chandrasekaran et al. [7] for SHM for so-called arborescence hypergraphs (see Section 2 for the definition). Even though arborescence hypergraphs include subpath hypergraphs, the results by Chandrasekaran et al. [7] do not imply any of our results, as they involve neither capacities nor optimization over stable matchings.

Organization. We start by introducing the necessary notation and formally define our studied computational problems in Section 2. We proceed with the University Dual Admission (UDA) problem in Section 3 as a practical example of SHbM. We move on to more abstract unimodular hypergraph classes in Section 4: in Section 4.1 we study the class of laminar hypergraphs, while in Section 4.2 we focus on the more general class of subpath hypergraphs. We examine subtree hypergraphs in Section 5, and finally conclude in Section 6.

Results marked with $(\star)$ have their proofs deferred to the full version of our paper [4].

2 Preliminaries

In Section 2.1 we define the concepts related to hypergraphs that we need in our paper, and in Section 2.2 we provide the definition of the computational problems we investigate.

2.1 Notation and Terminology

We use the notation $[\ell] = \{1, 2, \dots, \ell\}$ for each positive integer ℓ . We let $\mathbb{Z}, \mathbb{Z}_+, \mathbb{R}$, and $\mathbb{R}_+$ denote the set of integers, non-negative integers, reals, and non-negative reals, respectively. For the basic graph terminology and related standard notation we use, see the full version [4].

Matrices, polyhedra and Scarf's Lemma. For a matrix A , we let A_i denote its i th row. A matrix A is *totally unimodular* or *TU* if each subdeterminant of A has value in $\{-1, 0, 1\}$. Totally unimodular matrices are known for their useful properties. Most importantly, if A is

TU and b is integral, then all extreme points of the polyhedron $\{x \in \mathbb{R}^m : Ax \leq b, x \geq 0\}$ are integral. A point z in the polyhedron $\{x \in \mathbb{R}^m : Ax \leq b, x \geq 0\}$ is an *extreme point* if it cannot be written as a strictly convex combination of two different points in the polyhedron. That is, there are no $\lambda \in (0, 1)$ and $z_1, z_2 \in \{x \in \mathbb{R}^m : Ax \leq b, x \geq 0\}$ such that $z = \lambda z_1 + (1 - \lambda)z_2$.

A matrix A is a *network matrix* if it can be obtained from a directed graph $G = (V, E)$ and an edge set $F \subseteq E$ of G for which (V, F) is a spanning tree of G in the undirected sense, via the following method. With each row of A we associate an edge $f \in F$, and with each column we associate an edge $e \in E \setminus F$. Given edges $f \in F$ and $e \in E \setminus F$, let $C_{e,f}$ denote the unique cycle (in the undirected sense) contained in $(V, F \cup \{e\})$. Then, the entry of A at the intersection of row f and column e has value

- 1, if f is contained in $C_{e,f}$, and the edges e and f have the same orientation along $C_{e,f}$;
- -1, if f is contained in $C_{e,f}$, and e and f have a different orientation along $C_{e,f}$, and
- 0, if f is not contained in the cycle $C_{e,f}$.

It is well known that network matrices are totally unimodular (see [20, Theorem 13.20]).

The following is a key lemma of Scarf [19], useful for many stable matching problems.

► **Lemma 1** (Scarf [19]). *Let $A \in \mathbb{R}_+^{n \times m}$ be a matrix such that every column of A has a nonzero element, and let $b \in \mathbb{R}_+^n$. Suppose that every row $i \in [n]$ has a strict ordering $\succ_i$ over those columns $j \in [m]$ for which $A_{ij} > 0$. Then there exists an extreme point of $\{x \in \mathbb{R}^m : Ax \leq b, x \geq 0\}$ that dominates every column in some row, where we say that $x \in \mathbb{R}^m$ dominates column j in row i if $A_{ij} > 0$, $A_i x = b_i$, and $k \succeq_i j$ for all $k \in [m]$ such that $A_{ik} x_k > 0$. Also, such an extreme point can be found by a finite algorithm.*

Hypergraphs, matchings and b -matchings. A *hypergraph* $\mathcal{H} = (V, \mathcal{E})$ contains a set V of vertices and a set $\mathcal{E} \subseteq 2^V$ of *hyperedges*; we may simply refer to hyperedges as *edges* when this causes no confusion. We will say that a hyperedge $e \in \mathcal{E}$ is *incident* to a vertex $v \in V$ if $v \in e$. The *incidence matrix* of a hypergraph $\mathcal{H} = (V, \mathcal{E})$ has $|V|$ rows and $|\mathcal{E}|$ columns, and the entry at the intersection of column $v \in V$ and hyperedge $e \in \mathcal{E}$ is 1 if e is incident to v , and 0 otherwise. For a subset $F \subseteq \mathcal{E}$, let $F(v) := \{e \in F \mid v \in e\}$.

Given a hypergraph $\mathcal{H} = (V, \mathcal{E})$, a *matching* is a subset $M \subseteq \mathcal{E}$ that satisfies that $|M(v)| \leq 1$ for every $v \in V$. A matching M leaves a vertex v *unmatched* if $M(v) = \emptyset$, otherwise it *covers* v . Given *capacities* $b(v) \in \mathbb{Z}_+$ for each $v \in V$, we say that $M \subseteq \mathcal{E}$ is a *b -matching* if $|M(v)| \leq b(v)$ for each $v \in V$. For a b -matching M , we say that v is *unsaturated* in M if $|M(v)| < b(v)$, *saturated* if $|M(v)| = b(v)$, and *oversaturated* if $|M(v)| > b(v)$.

Hypergraph classes. Let us now describe the hypergraph classes relevant to our paper.

Let $\mathcal{H}$ be a hypergraph, and let $\mathcal{H}'$ be any *partial hypergraph* of $\mathcal{H}$, obtained by deleting some of its edges. The *chromatic index* of a hypergraph, denoted $\chi_e(\mathcal{H})$, is the smallest number of colors required to color its edges such that no two edges of the same color share a common vertex. It is clear that the maximum degree $\Delta(\mathcal{H})$ – the largest number of edges incident to any single vertex – provides a lower bound for $\chi_e(\mathcal{H})$.

► **Definition 2.** A hypergraph $\mathcal{H}$ is *normal* if $\chi_e(\mathcal{H}') = \Delta(\mathcal{H}')$ for every partial hypergraph $\mathcal{H}'$ of $\mathcal{H}$.

Lovász [15] provided an alternative characterization of normal hypergraphs.

► **Theorem 3** (Lovász). A hypergraph $\mathcal{H}$ is normal if and only if all extreme points of the polyhedron $\{x \in \mathbb{R}^m : Ax \leq 1, x \geq 0\}$ are integral, where A is the incidence matrix of $\mathcal{H}$.

An important subclass of normal hypergraphs is the class of unimodular hypergraphs.

► **Definition 4.** A hypergraph is unimodular if its incidence matrix is totally unimodular.

Next, we introduce subclasses of unimodular hypergraphs.

- A *network hypergraph* is a hypergraph whose incidence matrix is a network matrix.
- *Arborescence hypergraphs* are a subclass of network hypergraphs: $\mathcal{H} = (V, \mathcal{E})$ is an *arborescence hypergraph* if there is an arborescence F over V such that each edge of $\mathcal{E}$ forms a directed path in F .
- *Subpath hypergraphs* are the special case of arborescence hypergraphs where the underlying arborescence is required to be a path. Formally, $\mathcal{H} = (V, \mathcal{E})$ is a *subpath hypergraph* if there exists a directed path F over V such that each edge of $\mathcal{E}$ forms a subpath in F .
- A *laminar hypergraph* is a hypergraph $\mathcal{H} = (V, \mathcal{E})$ for which there are no two hyperedges $e_1, e_2 \in \mathcal{E}$ that satisfy $e_1 \setminus e_2 \neq \emptyset$ and $e_2 \setminus e_1 \neq \emptyset$. Laminar hypergraphs are known to be subpath hypergraphs; although this seems to be folklore, we provide a proof in the full version of our paper [4].

Finally, we define a superclass of arborescence hypergraphs. A hypergraph $\mathcal{H} = (V, \mathcal{E})$ is a *subtree hypergraph* if there is a tree $F = (V, E)$ on the ground set V such that each hyperedge induces a subtree of F . While subtree hypergraphs are not necessarily unimodular, they still form a subclass of normal hypergraphs [6, Section 4.4].

The full version of our paper [4] offers a full, graphical overview of the inclusion relations among these hypergraph classes.

2.2 Stable Hypergraph Matching

Let us start by defining the STABLE HYPERGRAPH MATCHING and STABLE HYPERGRAPH b -MATCHING problems. Let $\mathcal{H} = (V, \mathcal{E})$ be a hypergraph. For each $v \in V$ we are given a strict preference list $\succ_v$ over the set $\mathcal{E}(v)$ of hyperedges containing v . In the case of STABLE HYPERGRAPH b -MATCHING, we are also given capacities $b(v) \in \mathbb{Z}_+$ for each $v \in V$.

► **Definition 5.** A hyperedge $e \in \mathcal{E}$ is dominated by a (b) -matching M at some vertex $v \in V$ if v is saturated in M and $f \succ_v e$ for each hyperedge $f \in M(v)$. A hyperedge f blocks M if $f \notin M$ and there exists no vertex $v \in f$ at which M dominates f . A (b) -matching is stable if no hyperedge blocks it.

The most important question in such instances is to find a stable matching or b -matching. Hence, we get the following computational problem.

STABLE HYPERGRAPH b -MATCHING or SHbM

Input: A hypergraph $\mathcal{H} = (V, \mathcal{E})$ with capacities $b : V \rightarrow \mathbb{Z}$ and strict preferences $(\succ_v)_{v \in V}$ over incident edges for each vertex in V .

Output: Find a stable hypergraph b -matching in $\mathcal{H}$.

STABLE HYPERGRAPH MATCHING (or SHM) is defined analogously for matchings instead of b -matchings. It is clear that SHM is a subcase of SHbM where we have $b \equiv 1$.

For $\Pi \in \{\text{UNIMOD}, \text{SUBPATH}, \text{SUBTREE}\}$, we denote by Π -SHM and Π -SHbM the restrictions of SHM and SHbM to unimodular, subpath, or subtree hypergraphs, respectively.

As was observed by Aharoni and Fleiner [1], every stable hypergraph matching for an instance $(\mathcal{H}, (\succ_v)_{v \in V})$ of SHM with hypergraph $\mathcal{H} = (V, \mathcal{E})$ can be seen as an integral point in the polyhedron $\{x \in \mathbb{R}^m : Ax \leq 1, x \geq 0\}$ that dominates every column according

to the preferences $(\succ_v)_{v \in V}$, where A is the incidence matrix of $\mathcal{H}$. Thus, Scarf's Lemma and Theorem 3 imply the existence of a stable matching if the hypergraph is normal. Furthermore, for an instance $(\mathcal{H}, b, (\succ_v)_{v \in V})$ of SHbM, the integral points of the polyhedron $\{x \in \mathbb{R}^m : Ax \leq b, 0 \leq x \leq 1\}$ that dominate every column correspond to the stable hypergraph b -matchings, where b (with a slight abuse of notation) is the vector containing the capacities of the vertices. If the incidence matrix A is TU, then all extreme points of this polyhedron are integral, and thus Scarf's Lemma guarantees the existence of a stable b -matching whenever the underlying hypergraph is unimodular. The guaranteed existence of a stable matching or stable b -matching makes the class of normal hypergraphs and its subclass, unimodular hypergraphs, particularly interesting and useful.

Although the existence of a stable matching or b -matching may be guaranteed in restricted hypergraph classes, it is also of interest to find a maximum-weight stable hypergraph matching. For this, an optimization variant of SHbM (and SHM) can be formulated by associating a *weight* with each hyperedge. Given a weight function $w : \mathcal{E} \rightarrow \mathbb{Z}$ over a set $\mathcal{E}$ of hyperedges, we extend it by setting $w(M) = \sum_{e \in M} w(e)$ for each $M \subseteq \mathcal{E}$.

MAXIMUM-WEIGHT SHbM or MAXW-SHbM

Input: A hypergraph $\mathcal{H} = (V, \mathcal{E})$ with capacities $b : V \rightarrow \mathbb{Z}$, weights $w : \mathcal{E} \rightarrow \mathbb{Z}$, and strict preferences $(\succ_v)_{v \in V}$ over incident edges for each vertex in V .

Output: Find a maximum-weight stable hypergraph b -matching in $\mathcal{H}$.

We will refer to the optimization variant of the matching case, i.e., SHM as MAXIMUM-WEIGHT SHM or MAXW-SHM. For $\Pi \in \{\text{UNIMOD}, \text{SUBPATH}, \text{SUBTREE}\}$, we denote by $\Pi\text{-MAXW-SHM}$ and $\Pi\text{-MAXW-SHbM}$ the restrictions of MAXW-SHM and MAXW-SHbM to unimodular, subpath, or subtree hypergraphs, respectively.

We remark that in the special case when the hypergraph $\mathcal{H}$ is a bipartite graph, MAXW-SHbM coincides with the maximum-weight stable b -matching problem (MAXW-SbM) which is solvable in polynomial time [11].

3 The University Dual Admission Problem

In this section, we introduce a practically relevant problem in student allocation which, as we will show, can also be modeled as a special case of UNIMOD-SHM.

The problem is motivated by practical situations where certain programs in higher education are funded jointly as a result of cooperation between universities and companies, leading to a more complicated dual admission system. In our model, motivated by a real scenario in Hungarian higher education, we assume that such programs, funded by various companies, can be treated independently from each other (even though different programs might be funded by the same company) – hence, we shift the focus from the funding companies to the programs themselves.

Formally, we have a set $U = \{u_1, \dots, u_{n_U}\}$ of *universities*, a set $P_i = \{p_{i1}, \dots, p_{ik_i}\}$ of *programs* for each university $u_i \in U$, and a set $S = \{s_1, \dots, s_{n_S}\}$ of *students*. The set of programs is denoted by $P = \bigcup_{i \in [n_U]} P_i$. We may refer to the set of students, universities, and programs together as the set of *agents*. Each university $u_i \in U$ has a *capacity* $c(u_i)$, and each program $p_{ik} \in P$ has a *quota* $q(p_{ik})$.

The students apply to both a university $u_i \in U$ and a program $p_{ik} \in P_i$ available at university u_i . Since each program $p_{ik} \in P$ uniquely determines the university u_i offering it, this can alternatively be viewed as students applying simply to programs.¹ Thus, for each

¹ Nevertheless, universities play an important role in the admission problem, as will become clear in the

student $s_j \in S$ we assume a strict preference order $\succ_{s_j}$ over the set of programs acceptable for s_j . Additionally, each university $u_i \in U$ has a strict preference order $\succ_{u_i}$ over the students, and each program $p_{ik} \in P$ has a strict ordering $\succ_{p_{ik}}$ over the students. A triple (s_j, u_i, p_{ik}) is an *acceptable triple* if s_j finds p_{ik} acceptable; note that we do not consider acceptability for universities or programs explicitly, as we implicitly assume that a student finds a program acceptable only if both the program and university offering it finds the student acceptable as well. We may treat acceptable triples as subsets of $S \cup U \cup P$ of size 3.

An *assignment* μ is a function $\mu : S \rightarrow P \cup \{\perp\}$ mapping each student $s_j \in S$ either to a program that is acceptable for s_j , or to the special symbol $\perp$ meaning that s_j is left unassigned. For simplicity, we assume that a program is acceptable for s_j if and only if it is preferred by s_j to $\perp$. For a program $p_{ik} \in P$, we let $\mu(p_{ik})$ denote the set of students assigned to it by μ , and similarly, for a university $u_i \in U$ we let $\mu(u_i)$ denote the set of students assigned by μ to some program in P_i . An assignment μ is *feasible* if $|\mu(u_i)| \leq c(u_i)$ for each $u_i \in U$ and $|\mu(p_{ik})| \leq q(p_{ik})$ for each $p_{ik} \in P$; we say that a university or a program is *unsaturated* if the corresponding inequality is strict, otherwise it is *saturated*.

► **Definition 6.** A feasible assignment μ for an instance of UDA is stable if there is no blocking student–university–program triple, that is, a triple (s_j, u_i, p_{ik}) such that the following three conditions hold:

- (i) $p_{ik} \succ_{s_j} \mu(s_j)$;
- (ii) u_i satisfies one of the followings:
 - u_i is unsaturated,
 - there is a student $s_{j'} \in \mu(u_i)$ such that $s_j \succ_{u_i} s_{j'}$, or
 - $s_j \in \mu(u_i)$;
- (iii) the program p_{ik} is unsaturated, or there is a student $s_{j''} \in \mu(p_{ik})$ such that $s_j \succ_{p_{ik}} s_{j''}$.

We are ready to formally define the corresponding computational problem.

UNIVERSITY DUAL ADMISSION or UDA

Input: An instance $I = (S, U, P, c, q, (\succ_a)_{a \in S \cup U \cup P})$ of UDA as described above.

Question: Find a feasible and stable assignment for I .

Let us remark that the above problem can easily be used to model a situation where students may apply either to university–program pairs or only to universities: to allow for this option, we simply need to create a dummy program for each university $u_i \in U$ with quota $c(u_i)$ whose preferences are identical to the preferences of u_i .

A very natural idea to solve UDA is to use a Gale–Shapley-like proposal–rejection algorithm and iterate it until it outputs a stable matching. We give an example where a student-proposing variant of the Gale–Shapley algorithm goes into an infinite loop when used on an instance of UDA, even if students propose in a lexicographic order. It is also possible to create instances where universities or programs propose but the algorithm still cycles in a similar way. This suggests that the problem should be solved in a different manner.

3.1 Formulating UDA as a Special Case of SHbM

In this section, we first show that UDA can be formulated as a special case of the UNIMOD-SHbM, and as a consequence, a stable assignment always exists for every instance of UDA. By contrast, we also prove that a stable assignment of maximum size is NP-hard to find.

definition of a stable assignment (Definition 6).

► **Definition 7.** Given an instance I of UDA, let $\mathcal{H}_I = (S \cup U \cup P, \mathcal{E}_I)$ denote the hypergraph associated with I whose vertices are the agents in I and whose hyperedges are the acceptable triples in I . We say that a hypergraph $\mathcal{H}$ has the UDA property if it can arise as $\mathcal{H}_I$ for some instance I of UDA.

We call UDA-SHbM and UDA-MAXW-SHbM the restriction of SHbM and MAXW-SHbM to hypergraphs having the UDA property, respectively.

To create an instance of SHbM that is equivalent with our instance I of UDA, we also need to define a strict ranking over the hyperedges of $\mathcal{H}_I$ incident to each student, university, or program. For a student $s_j \in S$, his preference list $\succ_{s_j}$ in I can be viewed as a preference list over its incident hyperedges: s_j prefers a hyperedge $\{s_j, u_i, p_{ik}\}$ over $\{s_j, u_{i'}, p_{i'k'}\}$ if and only if $p_{ik} \succ_{s_j} p_{i'k'}$. For a program $p_{ik} \in P$, the preference order $\succ_{p_{ik}}$ similarly defines a ranking over the incident hyperedges, as each such hyperedge contains the same university u_i . For a university $u_i \in U$, we extend its preference list to all incident hyperedges the following way. We let u_i prefer $\{s_j, u_i, p_{ik}\}$ to $\{s_{j'}, u_i, p_{i'k'}\}$ if and only if either $s_j \succ_{u_i} s_{j'}$ or $s_j = s_{j'}$ and $p_{ik} \succ_{s_j} p_{i'k'}$. Let $\succ_a^{\mathcal{H}}$ denote the preferences of agent $a \in S \cup U \cup P$ in I thus defined.

Additionally, we define the capacity function $b^{\mathcal{H}} : S \cup U \cup P$ over the vertices of $\mathcal{H}_I$ as

$$b^{\mathcal{H}}(a) = \begin{cases} 1, & \text{if } a \in S, \\ c(u_i), & \text{if } a = u_i \in U, \\ q(p_{ik}), & \text{if } a = p_{ik} \in P. \end{cases}$$

► **Theorem 8** ($\star$). Stable assignments of an instance $I = (S, U, P, c, q, (\succ_a)_{a \in S \cup U \cup P})$ of UDA correspond bijectively to the stable $b^{\mathcal{H}}$ -matchings of the instance $I' = (\mathcal{H}_I, b^{\mathcal{H}}, (\succ_a^{\mathcal{H}})_{a \in S \cup U \cup P})$ of SHbM.

► **Theorem 9.** For each instance I of UDA, the associated hypergraph $\mathcal{H}_I$ is a network hypergraph.

Proof. We create a directed graph $D = (V, E)$ together with a spanning tree F in the undirected graph $\overline{D} = (V, \overline{E})$ obtained by replacing each arc (u, v) of D by an edge between u and v . We will show that the incidence matrix of $\mathcal{H}_I$ is exactly the network matrix that corresponds to D with the spanning tree F .

We create a vertex in V for each student, university, and program, and we further define an additional vertex x . Formally, we set $V = \{v(a) : a \in S \cup U \cup P\} \cup \{x\}$. The set E of arcs is defined as follows. First, for each program p_{ik} at some university u_i , we add an arc $e(p_{ik}) = (v(p_{ik}), v(u_i))$. Second, for each university u_i , we add an arc $e(u_i) = (v(u_i), x)$. Third, for each student s_j , we add an arc $e(s_j) = (x, v(s_j))$. Let $E_F = \{e(a) : a \in S \cup U \cup P\}$ denote the set of arcs defined so far. Finally, for each acceptable triple (s_j, u_i, p_{ik}) , i.e., for each hyperedge in $\mathcal{H}_I$, we add an arc $(v(s_j), v(p_{ik}))$. It is straightforward to see that $F := \{\{u, v\} : (u, v) \in E_F\}$ is a spanning tree in $\overline{D}$. Let A be the network matrix associated with D and F . Note that the edges of F are in a one-to-one correspondence with the vertices of the hypergraph $\mathcal{H}_I$, i.e., the set of agents in I . Furthermore, the edges of $E \setminus F$ are in a one-to-one correspondence with the hyperedges of $\mathcal{H}_I$.

Thus, it only remains to show that A is indeed the incidence matrix of $\mathcal{H}_I$, i.e., if a vertex $v \in V$ is contained in a hyperedge $\hat{e} \in \mathcal{E}_I$, then the corresponding entry in A is 1, and 0 otherwise. To see this, consider the unique cycle $C_{\hat{e}}$ defined by an arc $(v(s_j), v(p_{ik}))$ corresponding to some hyperedge $\hat{e} = \{s_j, u_i, p_{ik}\}$, which is the cycle formed by the four edges $(v(s_j), v(p_{ik}))$, $e(p_{ik})$, $e(u_i)$, and $e(s_j)$. Observe that $C_{\hat{e}}$ contains exactly three arcs of F , namely the arcs of D corresponding to the three vertices (i.e., agents) incident to $\hat{e}$. Hence, there are exactly three non-zero entries in the column of A corresponding to $\hat{e}$, and these

three entries are in the three rows that correspond to the vertices incident to $\hat{e}$. Moreover, since all arcs of $C_{\hat{e}}$ have the same orientation along $C_{\hat{e}}$, each of these three entries of A has value 1, as required. $\blacktriangleleft$

While Theorems 8 and 9 guarantee the existence of a stable assignment for every UDA instance, a maximum-size stable assignment is NP-hard to find, where the *size* of an assignment is the number of students it assigns to some program.

► **Theorem 10** ($\star$). *Finding a maximum-size stable assignment in an instance of UDA is NP-hard, even if all capacities are one.*

To handle this computational difficulty, we formulate an integer program whose solutions are exactly the stable assignments for an instance of UDA; see the full version [4].

3.2 An XP Algorithm for UDA-MAXW-SHbM

Let us describe an algorithm for an instance $I = (\mathcal{H}, b, w, (\succ_v)_{v \in V})$ of UDA-MAXW-SHbM, where $\mathcal{H} = (V, \mathcal{E})$ over vertex set $V = S \cup U \cup P$ is a hypergraph that satisfies the UDA property. Let $U = \{u_1, \dots, u_{n_U}\}$. We can assume that $P = \bigcup_{i \in [n_U]} P_i$, where the vertices $p \in P_i$ satisfy that for any $p \in e \in \mathcal{E}$, we have $\{u_i, p\} \subset e$.

The pseudocode of the algorithm is given in Algorithm 1. As a first step, we guess a strategy $\sigma : U \rightarrow \mathcal{E} \cup \{\perp\}$, that is, for each $u_i \in U$ we guess whether it will be saturated in the solution, and if so, what will be the worst hyperedge adjacent to u_i . Then, we create an instance I'_σ of MAXW-SbM, the maximum-weight bipartite stable b -matching problem. The underlying graph of I'_σ will be the bipartite graph $G_\sigma = (S \cup P, E_\sigma)$ where E_σ contains an edge $\{s, p\}$ if (i) there is a (unique) vertex $u \in U$ such that $e = \{s, u, p\} \in \mathcal{E}$ and (ii) for that vertex u , $e \succeq_u \sigma(u)$, where we assume that $e \succeq_u \perp$ always holds. The preferences of the agents in I'_σ are the projection of their original preferences. This is well defined, as each edge $\{s, p\} \in E_\sigma$ uniquely defines a hyperedge $\{s, u, p\} \in \mathcal{E}$; we further set $b'(s) = b(s)$, $b'(p) = b(p)$, and $w'(\{s, p\}) = w(\{s, u, p\})$ so that capacities and weights remain the same.

Let us say that a b' -matching M' in I'_σ is *valid* if, $M := \{\{s, u, p\} \in \mathcal{E} \mid \{s, p\} \in M'\}$ satisfies that for each $u \in U$, $|M(u)| = b(u)$ whenever $\sigma(u) \neq \perp$, and $|M(u)| < b(u)$ otherwise.

For a given strategy σ , the algorithm proceeds by computing a maximum-weight stable b' -matching M' in I'_σ . If M' is valid, then it adds the b -matching $M := \{\{s, u, p\} \in \mathcal{E} \mid \{s, p\} \in M'\}$ corresponding to M' to a set T . If M' is not valid, then we discard the guessed strategy σ . Finally, we output a maximum-weight stable b -matching from T .

► **Theorem 11.** *UDA-MAXW-SHbM can be solved in $\mathcal{O}((\Delta_U + 1)^{|U|}) \cdot \text{poly}(|\mathcal{E}|)$ time, where U correspond to the class of universities and Δ_U is the maximum degree of a vertex $u \in U$.*

Proof. We show that at the end of Algorithm 1, T will contain a maximum-weight stable b -matching for I .

▷ **Claim 12.** *If M' is a valid stable b' -matching in I'_σ , then the corresponding b -matching $M = \{\{s, u, p\} \in \mathcal{E} \mid \{s, p\} \in M'\}$ is feasible and stable in I .*

Proof. Let M' be a valid stable b' -matching in I'_σ and let M be defined as in the statement of the claim. Then, M is a feasible b -matching in I : for $v \in S \cup P$, $|M(v)| \leq b(v)$ follows from $b'(v) = b(v)$, and the capacities of the agents $u \in U$ are respected due to the validity of M' . It remains to show that M is stable in I .

For a hyperedge $e = \{s, u, p\}$, let $e_{I'} = \{s, p\}$. Suppose on the contrary that a hyperedge $e = \{s, u, p\}$ blocks M . First, if $e_{I'} \in E_\sigma$, then as p (respectively, s) is either unsaturated in both M and M' , or $e \succ_p f$ ($e \succ_s f'$) for some $f \in M(p)$ ($f' \in M(s)$) implying

■ **Algorithm 1** UDA-MAXW-SHbM($\mathcal{H} = (V, \mathcal{E}), b, w, (\succ_v)_{v \in V}$) where $\mathcal{H}$ defined over $S \sqcup U \sqcup P$ has the UDA property.

```

1:  $T := \emptyset$ 
2: for all strategies  $\sigma : U \rightarrow \mathcal{E} \cup \{\perp\}$  do
3:    $E_\sigma := \emptyset$ 
4:   for each  $e = \{s, u, p\} \in \mathcal{E}$  do
5:     if  $e \succeq_u \sigma(u)$  then  $E_\sigma := E_\sigma \cup \{\{s, p\}\}$ 
6:   Set  $I'_\sigma := ((S \cup P, E_\sigma), b', w', (\succ'_v)_{v \in S \cup P})$  where  $b'$ ,  $w'$ , and  $\succ'_v$  are obtained by
     projection from  $b$ ,  $w$ , and  $\succ_v$ , respectively.
7:   Compute a maximum-weight stable  $b'$ -matching  $M'$  in  $I'_\sigma$ .
8:    $M := \{\{s, u, p\} \in \mathcal{E} \mid \{s, p\} \in M'\}$ 
9:   if  $|M(u)| = b(u)$  whenever  $\sigma(u) \neq \perp$ , and  $|M(u)| < b(u)$  otherwise for all  $u \in U$  then
10:     $T := T \cup \{M\}$ 
11: return a maximum-weight  $b$ -matching  $M$  from  $T$ .

```

$e_{I'} \succ'_p f_{I'} \in M'$ ($e_{I'} \succ'_s f_{I'} \in M'$); this means that the edge $e_{I'}$ blocks M' , a contradiction. Second, if $\{s, p\} \notin E_\sigma$, then $\sigma(u) \succ_u e$. In particular, $\sigma(u) \neq \perp$ and due to the validity of M' we know that u is saturated in M . Moreover, by construction, any edge $\{s, p\} \in M'$ satisfies $\{s, u, p\} \succeq_u \sigma(u)$. Therefore, u is saturated in M with hyperedges that are not worse for u than $\sigma(u)$. Hence e cannot block M , a contradiction. $\triangleleft$

▷ **Claim 13.** Suppose that M is a stable b -matching in I . Define $\sigma : U \rightarrow \mathcal{E} \cup \{\perp\}$ such that $\sigma(u)$ is the worst hyperedge in $M(u)$ according to $\succ_u$ if u is saturated in M , and $\perp$ otherwise. Then every stable b' -matching M' in I'_σ is valid.

Proof. Let M' be the b' -matching in I'_σ defined as $M' = \{\{s, p\} : \{s, u, p\} \in M\}$. By the definition of σ , it holds that $M' \subseteq E_\sigma$, and moreover, M' is valid by construction. We claim that M' is stable in I'_σ .

To show that M' is stable in I'_σ , assume for the sake of contradiction that some edge $\{s, p\} \in E_\sigma$ blocks M' . Then, both s and p are either unsaturated or prefer $\{s, p\}$ to some edge in M' . Since M is stable, this means that the hyperedge $e = \{s, u, p\}$ must be dominated at u . Therefore, we get that u is saturated in M and $\sigma(u) \succ_u e$. However, then the edge $\{s, p\}$ cannot be contained in E_σ , a contradiction.

Due to the many-to-many extension of the well-known Rural Hospitals Theorem (see e.g., [11]), for any stable b' -matching M'' in I'_σ , we have $|M''(v)| = |M'(v)|$ for all $v \in S \cup P$. Hence, all stable b' -matchings of I'_σ must be valid, because for any stable b' -matching N' and its corresponding b -matching N in I , we have $|N(u_i)| = \sum_{p \in P_i} |N'(p)|$ for $u_i \in U$ by the UDA property. This concludes the proof. $\triangleleft$

As we know that a stable b -matching for I exists, Claim 13 guarantees that by iterating over all possible strategies, the algorithm is bound to find the strategy σ corresponding to the maximum-weight stable b -matching M . By Claims 12 and 13, $M' = \{\{s, p\} \mid \{s, u, p\} \in M\}$ will be a valid stable b' -matching for I'_σ , and since any valid b' -matching N' gives a stable b -matching N in I with $w(N') = w(N)$, the algorithm indeed puts M or a stable b -matching with the same weight in T .

Since there are at most $(\Delta_U + 1)^{|U|}$ possible strategies to check, and for each one them, the instance I'_σ and a maximum-weight stable b -matching in I'_σ can be computed in $\text{poly}(|\mathcal{E}|)$ time by a result of Fleiner [11], the claimed running time also follows. $\blacktriangleleft$

3.3 Further Results

On the positive side, we define a relaxed notion of stability in UDA, called *half-stability*, which intuitively means that for a triple to block, if the program is saturated, then the university and the program should be able to agree on a student that can be rejected; we show that we can find a half-stable assignment in polynomial time. See the full version of our paper [4] for further details.

► **Remark 14.** If $c(u) = 1$ for all universities, then a simple deferred acceptance algorithm between the students and universities produces a stable assignment; see the full version [4].

4 SHbM for Unimodular Hypergraphs

We proceed to study UNIMOD-SHbM in further classes of unimodular hypergraphs.

4.1 Laminar Hypergraphs

We construct a polynomial-time algorithm that finds a stable b -matching if the hypergraph $\mathcal{H} = (V, \mathcal{E})$ is laminar. As laminar hypergraphs are unimodular, the existence of a stable b -matching is guaranteed. We present our algorithm for this problem in Algorithm 2.

■ **Algorithm 2** Laminar-SHbM($\mathcal{H}, b, (\succ_v)_{v \in V}$) where $\mathcal{H} = (V, \mathcal{E})$ is laminar.

```

1:  $M := \emptyset$  ▷  $M$  is the current  $b$ -matching
2:  $\mathcal{C} := \emptyset$  ▷  $\mathcal{C}$  denotes the checked hyperedges
3: while  $\mathcal{E} \setminus \mathcal{C} \neq \emptyset$  do
4:   Pick an inclusion-wise minimal hyperedge  $f$  in  $\mathcal{E} \setminus \mathcal{C}$ .
5:    $\mathcal{C} := \mathcal{C} \cup \{f\}$ 
6:   if  $f$  blocks  $M$  then
7:     Set  $M := M \cup \{f\}$  and initialize  $X_f := \emptyset$ .
8:     for all  $v \in V$  with  $|M(v)| > b(v)$  do
9:       Add to  $X_f$  the hyperedge in  $M(v)$  that is worst for  $v$ .
10:    Set  $Y_f := \{e : e \text{ is inclusion-wise maximal in } X_f\}$  and  $M := M \setminus Y_f$ .
11: return  $M$ 

```

► **Theorem 15.** LAMINAR-SHbM is solved in polynomial time by Algorithm 2.

Proof. First, let us show that the set returned by Algorithm 2 is a b -matching. Observe that whenever some hyperedge f is added to M on line 7, the set X_f constructed on line 9 covers every vertex v that became oversaturated as a consequence of adding f . Therefore, Y_f also covers these vertices, so after the removal of all hyperedges of Y_f from the matching, the resulting set again satisfies all capacity constraints. Therefore, the algorithm indeed returns a b -matching M .

It remains to show that M is stable. Consider the iteration of lines 4–10 during Algorithm 2 where some hyperedge f is picked on line 4, and let M_f and M'_f denote the current b -matching at the beginning and at the end of this iteration, respectively. We need the following claim.

▷ **Claim 16.** For any edge f added to the b -matching at line 7, the set Y_f is a family of pairwise disjoint hyperedges whose union is a subset of f , with $f \notin Y_f$.

Proof. Since the algorithm always picks an inclusion-wise minimal hyperedge on line 4, the laminarity of $\mathcal{H}$ implies that every hyperedge that had been checked (i.e., put into $\mathcal{C}$) before f is either a subset of f or is disjoint from it. In particular, since the algorithm only puts hyperedges of the current b -matching into X_f , we get that all hyperedges in X_f are contained in f (recall that f shares at least one oversaturated vertex with each hyperedge in X_f). Since Y_f contains only the inclusion-wise maximal sets from X_f , the laminarity of $X_f \subseteq \mathcal{E}$ implies that the hyperedges in Y_f are pairwise disjoint.

To see $f \notin Y_f$, it suffices to observe the following. Since f blocks the current b -matching M_f , for each $v \in V$ that is oversaturated in $M_f \cup \{f\}$ we know that f cannot be the worst hyperedge incident to v in $M_f(v) \cup \{f\}$. $\triangleleft$

Consider now the iteration during which f is picked on line 4. If the algorithm adds f to the current b -matching M_f on line 4, then $M'_f = M_f \cup \{f\} \setminus Y_f$, otherwise $M'_f = M_f$. Hence, the above claim implies $|M'_f(v)| \geq |M_f(v)|$ for each $v \in V$, since v can only be incident to at most one hyperedge in Y_f . Moreover, if $|M_f(v)| = b(v)$ for some $v \in V$, then the worst hyperedge in $M'_f(v)$ is weakly preferred by v to the worst hyperedge in $M_f(v)$: this is obvious if $v \notin f$, and if $v \in f$, then v prefers f to at least one hyperedge in $M_f(v)$, so v 's worst hyperedge cannot be worse than before, as f is the only new hyperedge in $M'_f(v)$ not in $M_f(v)$ already. Hence, whenever some vertex becomes saturated with hyperedges better than some $e \in \mathcal{E}$, then this remains true during the remainder of the algorithm. We refer to this observation as the *saturation-monotonicity* of the algorithm.

To prove the stability of M , consider an arbitrary edge $f \in \mathcal{E} \setminus M$ and the iteration during which f is examined at line 4. If f is not added to M_f on line 7, then some vertex v incident to f must be saturated in M_f with hyperedges that are all preferred to f by v . By the saturation-monotonicity of the algorithm, this remains true for the returned b -matching M , so f cannot block M . By contrast, if f is added to M_f on line 7, then f must have been removed at some point on line 10 during a later iteration as a consequence of adding some hyperedge e to the b -matching M_e on line 7 for which $f \in Y_e$. By our construction of Y_e , we know that f is the worst hyperedge for some vertex $u \in V$ in $M_e(u)$, so after removing Y_e from the b -matching $M_e \cup \{e\}$, vertex u is saturated with hyperedges that are all preferred by u to f . Again, by the saturation-monotonicity of the algorithm, this remains true for u during the algorithm, so f cannot block M . Thus, M is indeed a stable b -matching for $\mathcal{H}$.

In every iteration, a hyperedge of $\mathcal{E} \setminus \mathcal{C}$ gets added to $\mathcal{C}$, so there are $|\mathcal{E}|$ iterations. As each iteration can be performed in linear time, the running time is polynomial. $\blacktriangleleft$

We complement Theorem 15 by a hardness result for a very restricted weight function.

► **Theorem 17** (*). *The MAXW-LAMINAR-SHM problem is NP-complete on laminar hypergraphs, even if all edges have weight 0 with the exception of a single edge of weight 1.*

4.2 Subpath Hypergraphs

Moving our focus to instances of SHbM where the underlying hypergraph $\mathcal{H}$ is a subpath hypergraph, we present an algorithm for the MAXW-SUBPATH-SHbM problem that runs in polynomial time if the maximum size of a hyperedge in $\mathcal{H}$ is bounded by a constant.

► **Theorem 18.** *MAXW-SUBPATH-SHbM can be solved in $O((\Delta(b_{\max} + 1))^{\ell_{\max}} |\mathcal{E}| \log \omega)$ time for an instance $(\mathcal{H} = (V, \mathcal{E}), b, w, (\succ_v)_{v \in V})$, where $b_{\max}$ is the maximum capacity of a vertex, $\ell_{\max}$ is the maximum size of an edge in $\mathcal{E}$, Δ is the maximum degree of a vertex, and ω is the maximum absolute value of $w(e)$ over $e \in \mathcal{E}$.*

A brute-force approach to solve the MAXW-SUBPATH-SHbM problem would be to consider the edges one-by-one and enumerate all possible b -matchings that do not violate the capacities and that may turn out to be stable in the end. However, such a brute-force method would not yield a polynomial-time algorithm even if $\ell_{\max}$ is constant. Hence, we apply a more sophisticated approach using dynamic programming: we process edges one-by-one, but instead of recording all possible partial solutions, we only maintain a smaller set of b -matchings that, in a certain sense, represents all possible strategies for obtaining a stable b -matching. We start by introducing the necessary concepts.

Strategies. A *strategy* over a given set $W \subseteq V$ of vertices is a function $\sigma : W \rightarrow \mathcal{E} \cup \{\perp\}$ such that $\sigma(w)$ is incident to w for each $w \in W$ unless $\sigma(w) = \perp$. The interpretation of a strategy is that $\sigma(w)$ is the planned *threshold* of w , that is, the worst edge that we plan to put into the b -matching among those incident to w ; the symbol $\perp$ corresponds to being unsaturated. We extend the notation $\succ_w$ such that $e \succ_w \perp$ holds for each e incident to w .

► **Definition 19.** A b -matching M realizes a strategy σ over W w.r.t. some $\mathcal{E}' \subseteq \mathcal{E}$ if

- (i) M is compatible with σ , meaning that for each $w \in W$:
 - if $\sigma(w) \in \mathcal{E}$, then $e \succeq_w \sigma(w)$ holds for every $e \in M(w)$, and
 - if $\sigma(w) = \perp$, then $|M(w)| < c(w)$;
- (ii) $\sigma(w) \in M(w)$ for each $w \in W$ with $\sigma(w) \in \mathcal{E}'$, and
- (iii) every edge in $\mathcal{E}' \setminus M$ that blocks M is planned to be dominated by σ where we say that σ plans to dominate an edge e if there exists some $w \in W$ such that $\sigma(w) \succ_w e$.

We call a strategy *realizable* for an edge set $\mathcal{E}' \subseteq \mathcal{E}$ if some b -matching realizes it w.r.t. $\mathcal{E}'$.

Observe that if a strategy σ plans to dominate an edge e at some vertex w and M is a b -matching compatible with σ that also satisfies $|M(w)| = b(w)$, then e cannot block M , because e is *dominated* by M at w , meaning that M contains $b(w)$ edges incident to w , and w prefers each of them to e .

Given a strategy σ over W and a b -matching compatible with σ , we say that M is *complete* on a vertex set $W' \subseteq W$ for σ if $|M(v)| = b(v)$ for each $v \in W'$ such that $\sigma(v) \neq \perp$.

The algorithm. Let $(\mathcal{H}, b, w, (\succ_v)_{v \in V})$ be our instance of MAXW-SUBPATH-SHbM with a subpath hypergraph $\mathcal{H} = (V, \mathcal{E})$, and let $v_1, \dots, v_n$ be the ordering of V witnessing the subpath property of $\mathcal{H}$. That is, for each hyperedge $e \in \mathcal{E}$ there exist indices $i \leq j$ such that $e = \{v_i, v_{i+1}, \dots, v_j\}$.

For a hyperedge $e \in \mathcal{E}$, we write $l(e) = \max\{i : v_i \in e\}$ and we say that $v_{l(e)}$ is the *endpoint* of e . Let $e_1, \dots, e_m$ be an ordering of $\mathcal{E}$ where $l(e_i) < l(e_j)$ implies $i < j$, that is, we order the edges increasingly according to their endpoints. Let $\mathcal{E}_i = \{e_1, \dots, e_i\}$, and let L_i contain the last $\ell_{\max}$ vertices contained in $\bigcup \mathcal{E}_i$, that is, $L_i = \{v_j : \max\{0, l(e_i) - \ell_{\max}\} < j \leq l(e_i)\}$.

Our algorithm considers each set $\mathcal{E}_i = \{e_1, \dots, e_i\}$, and computes a set S_i of pairs (σ, M) where σ is a strategy over L_i and M is a b -matching in $(V, \mathcal{E}_i)$ that realizes σ . We will ensure that S_i is *representative* for $\mathcal{E}_i$ in the following sense:

► **Definition 20.** Set S_i is w -representative for $\mathcal{E}_i$ if the following holds: whenever there is a strategy σ and a b -matching M realizing σ with respect to $\mathcal{E}_i$, then there is a b -matching M' equivalent to M on L_i for which $(\sigma, M') \in S_i$ and $w(M') \geq w(M)$. Here, M is equivalent to M' on L_i , if $|M(v)| = |M'(v)|$ for each $v \in L_i$; this is denoted by $M \sim_{L_i} M'$. The operation of updating S_i with (σ, M) means that we put (σ, M) into S_i unless there is some pair $(\sigma, \hat{M})$ already in S_i for which $\hat{M}$ is equivalent on L_i to M and $w(\hat{M}) \geq w(M)$.

We initialize S_0 to contain only $(\sigma_\emptyset, \emptyset)$ where $\sigma_\emptyset$ is the function with empty domain. For $i = 1$ to m , we build the set S_i as follows; Algorithm 3 contains a formal description.

Adjusting the domain from L_{i-1} to L_i : For each $(\sigma, M) \in S_{i-1}$, check whether M is complete for σ on $L_{i-1} \setminus L_i$. If not, then we will not be able to make σ complete by adding edges from $\mathcal{E} \setminus \mathcal{E}_{i-1}$, so we ignore the pair (σ, M) . Otherwise, we change the domain of σ from L_{i-1} to L_i (forgetting all values assigned to vertices of $L_{i-1} \setminus L_i$) as follows: We set $\sigma(v)$ for each $v \in L_i \setminus L_{i-1}$ to all possible values in $\{e \in \mathcal{E} : v \in e\} \cup \{\perp\}$, and for each strategy σ' over L_i obtained this way, we put (σ', M) into a set T_i .

Computing S_i based on e_i : For each $(\sigma, M) \in T_i$ we check whether M still realizes σ with respect to $\mathcal{E}_i$ by checking if (i) σ plans to dominate e_i , and (ii) for each $v \in e_i$, the threshold $\sigma(v)$ does not coincide with e_i . If these conditions hold, then M realizes σ with respect to $\mathcal{E}_i$. Hence, we update S_i with (σ, M) . Next, we check whether we can add e_i to M : if adding e_i does not violate the capacities and yields a b -matching compatible with σ , then we update S_i with $(\sigma, M \cup \{e_i\})$.

The algorithm finishes by checking whether S_m contains a pair (σ, M) where M is complete for σ on L_m , and if so, outputs among all such b -matchings M one that maximizes $w(M)$.

■ **Algorithm 3** MAXW-SUBPATH-SHbM($\mathcal{H} = (V, \mathcal{E}), b, w, (\succ_v)_{v \in V}$) where $\mathcal{H}$ is a subpath hypergraph.

```

1: Set  $L_0 := \emptyset$  and  $\sigma_\emptyset$  as the empty function over  $\emptyset$ .
2: Set  $S_0 := \{(\sigma_\emptyset, \emptyset)\}$ .
3: for  $i = 1$  to  $m$  do
4:   for all pair  $(\sigma, M) \in S_{i-1}$  do
5:     if  $M$  is not complete for  $\sigma$  on  $L_{i-1} \setminus L_i$  then
6:       break; ▷ Continue with next pair in  $S_{i-1}$ .
7:     for all  $\sigma'' : L_i \setminus L_{i-1} \rightarrow \mathcal{E} \cup \{\perp\}$  do
8:       if for each  $v \in L_i \setminus L_{i-1}$ , either  $\sigma''(v) \ni v$  or  $\sigma''(v) = \perp$  then
9:         Set  $\sigma'$  as the union  $\sigma|_{L_{i-1} \cap L_i}$  and  $\sigma''$ , with domain  $L_i$ .
10:       $T_i \leftarrow (\sigma', M)$ 
11:    $S_i := \emptyset$ 
12:   for all pair  $(\sigma, M) \in T_i$  do
13:     if  $\sigma$  plans to dominate  $e_i$  and  $e_i \neq \sigma(v)$  for each  $v \in e_i$  then
14:       if  $\exists(\sigma, \hat{M}) \in S_i : M \sim_{L_i} \hat{M}$  and  $w(\hat{M}) \geq w(M)$  then  $S_i \leftarrow (\sigma, M)$ 
15:     if  $e_i \succeq_v \sigma(v)$  for each  $v \in e_i$  and  $\forall v \in e_i : |M(v)| < b(v)$  then
16:        $M' := M \cup \{e_i\}$ 
17:       if  $\exists(\sigma, \hat{M}) \in S_i : M' \sim_{L_i} \hat{M}$  and  $w(\hat{M}) \geq w(M')$  then  $S_i \leftarrow (\sigma, M')$ 
18: Set  $w^* := -\infty$  and  $M^* := \emptyset$ .
19: for all pair  $(\sigma, M) \in S_m$  do
20:   if  $M$  is complete for  $\sigma$  on  $L_m$  and  $w(M) > w^*$  then
21:     Set  $w^* := w(M)$  and  $M^* := M$ .
22: return  $M^*$ .

```

► **Lemma 21** (*). *If Algorithm 3 puts a pair (σ, M) into S_i for some $i \in [m]$, then M realizes σ with respect to $\mathcal{E}_i$.*

Proof sketch. We use induction on i . The case $i = 0$ is trivial, so assume that the lemma holds for $i - 1$.

First, suppose that (σ, M) was added to S_i on line 14, so $M \subseteq \mathcal{E}_{i-1}$. Let $(\sigma^-, M) \in S_{i-1}$ be selected on line 4 in the cycle during which (σ, M) was put into T_i on line 10. By induction,

- (a) M is compatible with σ^- ,
- (b) $\sigma^-(v) \in M(v)$ for every $v \in L_{i-1}$ where $\sigma^-(v) \in \mathcal{E}_{i-1}$, and
- (c) every edge in $\mathcal{E}_{i-1} \setminus M$ that blocks M is planned to be dominated by σ^- .

First, observe that M is compatible with σ : the compatibility conditions for each $v \in L_{i-1} \cap L_i$ follow immediately from $\sigma^-(v) = \sigma(v)$ due to (a), while the compatibility condition for each $v \in L_i \setminus L_{i-1}$ holds irrespective of the value of $\sigma(v)$, because edges of $M \subseteq \mathcal{E}_{i-1}$ have no vertices in $L_i \setminus L_{i-1}$.

Second, by our assumption that Algorithm 3 added (σ, M) to S_i on line 14, we know that $\sigma(v) \neq e_i$ for each v incident to e_i . Hence, if $\sigma(v) \in \mathcal{E}_i$ for some $v \in L_i$, then $\sigma(v) \in \mathcal{E}_{i-1}$, and consequently, $v \in L_i \cap L_{i-1}$ (because vertices of $L_i \setminus L_{i-1}$ are not contained in any edge of $\mathcal{E}_{i-1}$). Therefore, (b) implies $\sigma(v) = \sigma^-(v) \in M(v)$. Third, it remains to show that every edge $e \in \mathcal{E}_i \setminus M$ that blocks M is planned to be dominated by σ . First note that Algorithm 3 checks that this holds for e_i explicitly on line 13, so we may assume $e \neq e_i$. By (c), we know that every edge in $\mathcal{E}_{i-1} \setminus M$ that blocks M is planned to be dominated by σ^- , so there is some $v \in L_{i-1}$ such that $\sigma^-(v) \succ_v e$. On the one hand, if $v \in L_i \cap L_{i-1}$, then e is planned to be dominated by σ too, because $\sigma(v) = \sigma^-(v)$. On the other hand, if $v \in L_{i-1} \setminus L_i$, then Algorithm 3 must have confirmed on line 10 that M is complete on v (otherwise it would not have put (σ^-, M) into T_i), which implies that M dominates e at v . Thus, e does not block M , proving that every edge in $\mathcal{E}_i \setminus M$ that blocks M is planned to be dominated by σ . This finishes the proof that M realizes σ with respect to $\mathcal{E}_i$.

The case when (σ, M) was put into S_i on line 17 is similar; see the full version [4]. ◀

► **Lemma 22** ($\star$). *Set S_i computed by Algorithm 3 is w -representative for $\mathcal{E}_i$ for each $i \in [m]$.*

Proof sketch. We prove the lemma by induction on i . Clearly, the lemma trivially holds for $i = 0$, so let us assume that the statement holds for $i - 1$.

To prove that S_i remains w -representative for $\mathcal{E}_i$, it suffices to show the following: if (σ, M) is such that $M \subseteq \mathcal{E}_i$ realizes σ with respect to $\mathcal{E}_i$, then the algorithm updates S_i with *some* pair (σ, M') either at line 14 or at line 17 for which $M' \sim_{L_i} M$ (note that $\sim_{L_i}$ is transitive) and $w(M') \geq w(M)$.

Given σ and M , let us define the strategy σ^- over L_{i-1} as follows:

$$\sigma^-(v) = \begin{cases} \sigma(v), & \text{if } v \in L_{i-1} \cap L_i; \\ \perp, & \text{if } v \in L_{i-1} \setminus L_i \text{ and } |M(v)| < b(v); \\ \text{worst}(M, v), & \text{if } v \in L_{i-1} \setminus L_i \text{ and } |M(v)| = b(v) \end{cases}$$

where $\text{worst}(M, v)$ is the worst edge in $M(v)$ according to $\succ_v$. Note that M is complete on vertices of $L_{i-1} \setminus L_i$ for σ^- . We distinguish between two cases.

Case A: $e_i \notin M$. It is then straightforward to check that M realizes σ^- w.r.t. $\mathcal{E}_{i-1}$; see [4] for a detailed proof. Therefore, by induction there exists a b -matching M' with $w(M') \geq w(M)$ such that $(\sigma^-, M') \in S_{i-1}$ and $M' \sim_{L_{i-1}} M$.

Recall that M is complete on $L_{i-1} \setminus L_i$ for σ^- , and thus $M' \sim_{L_{i-1}} M$ implies that M' is also complete on $L_{i-1} \setminus L_i$ for σ^- . Hence, (σ^-, M') passes the test on line 5. As Algorithm 3 tries all possible ways to extend σ^- , it follows that $(\sigma, M') \in T_i$. Now, since M realizes σ w.r.t. $\mathcal{E}_i$ but $e_i \notin M$, we get that $e_i \neq \sigma(v)$ for any $v \in e_i$. Additionally, σ plans to dominate e_i : if e_i blocks M , then this is because M realizes σ w.r.t. $\mathcal{E}_i$ (by our assumptions), and if e_i does not block M , then it is dominated at some vertex $v \in L_i$, which in turn implies that v is saturated by M and $\text{worst}(M, v) = \sigma(v) \succ_v e_i$, that is, σ indeed plans to dominate e_i . Therefore, Algorithm 3 updates S_i with (σ, M') at line 14.

Case B: $e_i \in M$. Consider the b -matching $M^- = M \setminus \{e_i\}$. It is then straightforward to check that M^- realizes σ^- w.r.t. $\mathcal{E}_{i-1}$; see again [4]. Therefore, by induction there exists a b -matching M' with $w(M') \geq w(M^-)$ such that $(\sigma^-, M') \in S_{i-1}$ and $M' \sim_{L_{i-1}} M^-$.

Recall that M^- is complete on $L_{i-1} \setminus L_i$ for σ^- , and thus $M' \sim_{L_{i-1}} M^-$ implies that M' is also complete on $L_{i-1} \setminus L_i$ for σ^- . Therefore, (σ^-, M') passes the test on line 5. As Algorithm 3 tries all possible ways to extend σ^- , it follows that $(\sigma, M') \in T_i$. Since $e_i \in M$ and M is compatible with σ , we know that $e_i \succeq_v \sigma(v)$ for each $v \in L_i$. By $M' \sim_{L_{i-1}} M^-$ and since $M^- \cup \{e_i\} = M$ is a b -matching, we know $|M'(v)| < b(v)$ for each $v \in e_i \cap L_{i-1}$; since $M' \subseteq \mathcal{E}_{i-1}$, it is also clear that $|M'(v)| = 0 < b(v)$ for each $v \in L_i \setminus L_{i-1}$. Therefore, the pair (σ, M') satisfies the conditions on line 15 and so Algorithm 3 updates S_i with $(\sigma, M' \cup \{e_i\})$ at line 17. By $M' \cup \{e_i\} \sim_{L_i} M^- \cup \{e_i\} = M$ and $w(M' \cup \{e_i\}) \geq w(M^-) + w(e_i) = w(M)$, this finishes the proof of the lemma. $\blacktriangleleft$

Proof of Theorem 18. Let us first prove that Algorithm 3 solves MAXW-SUBPATH-SHbM correctly on subpath hypergraphs.

First, we show that the output is stable. Observe that if Algorithm 3 returns a b -matching M on line 22, then M realizes σ on $\mathcal{E}$, and thus σ plans to dominate every edge blocking M . However, as M is complete for σ (due to the condition on line 20), M in fact dominates every edge that σ plans to dominate. Thus, no edge may block M .

Assume now that M is a maximum-weight stable b -matching in our instance. Let σ be the strategy over L_m defined as

$$\sigma(v) = \begin{cases} \perp, & \text{if } |M(v)| < b(v); \\ \text{worst}(M, v), & \text{if } |M(v)| = b(v). \end{cases}$$

It is clear that M realizes σ over $\mathcal{E}$. Thus, by Lemma 22 there exists a b -matching M' satisfying $w(M') \geq w(M)$ for which $(\sigma, M') \in S_m$ and $M \sim_{L_m} M'$. Moreover, as M is complete on L_m for σ and $M \sim_{L_m} M'$, we know that M' is complete on L_m for σ as well. Hence, Algorithm 3 will set w^* to at least $w(M') \geq w(M)$ on line 21 at latest when it examines the pair (σ, M') , and will thus return a stable b -matching of weight at least $w(M)$ on line 22, which must be a maximum-weight stable b -matching.

To bound the running time of Algorithm 3, note that the number of possible strategies for a given set L_i is at most $\Delta^{|L_i|} \leq \Delta^{\ell_{\max}}$ where Δ is the maximum number of edges incident to any vertex. Given a strategy σ over L_i , we also know $|\{M : (\sigma, M) \in S_i\}| \leq (b_{\max} + 1)^{|L_i|} \leq (b_{\max} + 1)^{\ell_{\max}}$ since this set does not contain two b -matchings that are equivalent on L_i . This yields

$$|S_i| \leq (\Delta(b_{\max} + 1))^{\ell_{\max}}.$$

There are m iterations, and the computation of S_i from S_{i-1} is proportional to $|S_{i-1}| + |S_i|$, as any line of the algorithm can be performed in $O(\ell_{\max} \log \omega)$ time. Thus, we obtain that the total running time is $O((\Delta(b_{\max} + 1))^{\ell_{\max}} m \log \omega)$, as promised. $\blacktriangleleft$

► **Remark 23.** We remark that Algorithm 3 can be used to decide whether a stable b -matching exists that contains a given set of “forced” edges (or is disjoint from a given set “forbidden” edges) by setting an appropriate weight function. Furthermore, it can also be used to decide the existence of a stable b -matching that saturates (or does not saturate) a given vertex v . Indeed, to decide whether some stable b -matching saturates v , it suffices to add a newly created hyperedge e_v incident only to v , extend the preferences of v such that e_v becomes the least-favorite edge according to $\succ_v$, and decide whether this modified instance admits a stable b -matching that does not contain e_v . Deciding whether some stable b -matching leaves v unsaturated can be done analogously, using the same construction but searching for a stable b -matching that contains e_v .

■ **Algorithm 4** Subtree-SHM($\mathcal{H}, (\succ_v)_{v \in V}$) where $\mathcal{H} = (V, \mathcal{E})$ is a subtree hypergraph with underlying tree T rooted at $r \in V$.

```

1: if  $\mathcal{E} = \emptyset$  then return  $M = \emptyset$ .
2:  $f := \operatorname{argmax}_{e \in \mathcal{E}} \operatorname{dist}_{T,r}(e)$ 
3:  $r_f := \operatorname{top}_{T,r}(f)$ 
4:  $X_f := \{f' \in \mathcal{E} \mid r_f \in f', f \succ_{r_f} f'\}$ 
5:  $\mathcal{E}' := \mathcal{E} \setminus (X_f \cup \{f\})$ 
6:  $M = \text{Subtree-SHM}((V, \mathcal{E}'), (\succ_v)_{v \in V})$ 
7: if  $r_f$  is not covered by  $M$  then  $M := M \cup \{f\}$ .
8: return  $M$ .

```

Theorem 24 shows that the running time of the algorithm in Theorem 18 cannot be improved to an FPT algorithm with parameter $\ell_{\max}$, not even if we additionally parameterize the problem with the maximum capacity and severely restrict the weight function.

► **Theorem 24** (*). *The MAXW-SUBPATH-SHbM problem is W[1]-hard when parameterized by $b_{\max} + \ell_{\max}$, where $b_{\max}$ is the maximum capacity of a vertex and $\ell_{\max}$ is the maximum size of an edge, even if all edges have weight 0 with the exception of a single edge of weight 1.*

5 SHbM for Subtree Hypergraphs

We now turn our focus to the class of subtree hypergraphs. As these are not necessarily unimodular, so an instance of SUBTREE-SHbM may not admit a stable b -matching. However, as subtree hypergraphs are still normal, SUBTREE-SHM is guaranteed to be solvable.

We start by presenting a polynomial-time algorithm that solves SUBTREE-SHM. Before explaining our algorithm, let us introduce some additional notation. Let $\mathcal{H} = (V, \mathcal{E})$ be a subtree hypergraph and let $T = (V, F)$ be the underlying tree, that is, for each hyperedge $e \in \mathcal{E}$, the set of vertices contained in e induces a subtree of T . Let r be a fixed vertex of V and consider the rooted tree T with root r . For each vertex v , let $\operatorname{dist}_T(r, v)$ be the *distance* of r and v in T , that is the number of edges of the (unique) path of T between r and v . For each hyperedge $e \in \mathcal{E}$, let $\operatorname{top}_{T,r}(e)$ be the (unique) vertex of e that is closest to r in T , and define $\operatorname{dist}_{T,r}(e) := \operatorname{dist}_T(r, \operatorname{top}_{T,r}(e))$.

Given an instance $(\mathcal{H}, (\succ_v)_{v \in V})$ of SUBTREE-SHM, where $\mathcal{H} = (V, \mathcal{E})$ is a subtree hypergraph with underlying tree T , algorithm Subtree-SHM does the following (see Algorithm 4 for a pseudocode). First, it finds a hyperedge f such that $\operatorname{top}_{T,r}(f)$ is furthest from r in T ; for simplicity, let us write r_f for $\operatorname{top}_{T,r}(f)$. Then it deletes f and every hyperedge incident to r_f that are less preferred by r_f than f , and recursively calls itself on the remaining hypergraph. Finally, it builds a matching by adding f to the output M of the recursive call if r_f is not covered by M .

► **Theorem 25.** *Algorithm 4 solves SUBTREE-SHM in polynomial time.*

Proof. The algorithm runs in polynomial time, because Subtree-SHM is called at most $|\mathcal{E}|$ times recursively, and in each call, it only needs to compute the farthest hyperedge f and delete those hyperedges incident to $r_f = \operatorname{top}_{T,r}(f)$ that are worse than f for r_f .

We show that the output M is a stable hypergraph matching. We use induction on $|\mathcal{E}|$. If $\mathcal{E} = \emptyset$, then the output $M = \emptyset$ of Algorithm 4 is clearly a stable hypergraph matching.

Hence, assume that $\mathcal{E} \neq \emptyset$. Let f be the hyperedge chosen by the algorithm in the first iteration. By induction, M is a stable hypergraph matching in the subtree hypergraph $(V, \mathcal{E} \setminus (X_f \cup \{f\}))$.

First, suppose that M saturates r_f , so Algorithm 4 outputs M . Then, M is clearly a matching. Also, no hyperedge of $\mathcal{E} \setminus (X_f \cup \{f\})$ blocks M by induction. Finally, no hyperedge of $X_f \cup \{f\}$ blocks M either, because r_f is covered with some hyperedge $e \in M$ such that $e \succ_{r_f} f \succ_{r_f} f'$ for every $f' \in X_f$ by the choice of X_f .

Suppose now that M does not saturate r_f , and the output is therefore $M \cup \{f\}$. The fact that $\mathcal{H}$ is a subtree hypergraph and that f is chosen so that it maximizes $\text{dist}_{T,r}(f)$ implies that any hyperedge that intersects f must also contain r_f . Hence, as M does not cover r_f , $M \cup \{f\}$ is a matching. By induction, no hyperedge of $\mathcal{E} \setminus (X_f \cup \{f\})$ blocks M , and thus $M \cup \{f\}$. Finally, no hyperedge of X_f blocks $M \cup \{f\}$, because $f \succ_{r_f} f'$ for every $f' \in X_f$ and r_f is saturated by $M \cup \{f\}$.

We conclude that the output is a stable matching. $\blacktriangleleft$

A natural question to ask whether a stable b -matching can still be found efficiently in the SHbM setting. Sadly, the answer is no, even with very severe restrictions.

► **Theorem 26** (*). *Deciding if there exists an stable hypergraph b -matching in an instance of SUBTREE-SHbM is NP-hard, even if the underlying tree is a star, only one vertex has capacity greater than 1, and all hyperedge sizes are at most 4.*

► **Remark 27.** Using the reduction presented in the proof of Theorem 26 and the fact that the problem of finding a stable *fractional* hypergraph matching is PPAD-complete even if each hyperedge has size 3 [8], it is straightforward to verify that the problem of finding a stable fractional hypergraph matching in an instance of SUBTREE-SHbM satisfying the conditions of Theorem 26 is PPAD-complete too.

6 Conclusions

We studied stable hypergraph matchings and b -matchings, focusing on structured hypergraph families. We introduced the University Dual Admission problem and showed that it can be regarded as a special case of UNIMOD-SHbM. We identified multiple classes of hypergraphs where stable (b)-matchings can be found efficiently, and provided contrasting proofs of intractability; see Table 1 for a summary.

Besides some open questions in Table 1, the main problem we leave open for further research is whether it is possible to find a stable matching in an arbitrary instance of UNIMOD-SHM in polynomial time. Our complexity results seem to suggest that the problem might be computationally hard (e.g., PPAD-hard) in general; however, for special cases there is a potential for efficient combinatorial algorithms. In particular, resolving the complexity of this problem for network hypergraphs would be of great interest. More generally, the development of new techniques to attain stable solutions for instances of SHM and SHbM is an important direction for future research that is motivated by various practical application such as the HOSPITALS/RESIDENTS WITH COUPLES problem [16, Chapter 5.3].

References

- 1 Ron Aharoni and Tamás Fleiner. On a lemma of Scarf. *Journal of Combinatorial Theory, Series B*, 87(1):72–80, 2003. doi:10.1016/S0095-8956(02)00028-X.
- 2 Haris Aziz and Florian Brandl. Existence of stability in hedonic coalition formation games. In *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2012)*, volume 2, pages 763–770, 2012. URL: <http://dl.acm.org/citation.cfm?id=2343806>.

- 3 Suryapratim Banerjee, Hideo Konishi, and Tayfun Sönmez. Core in a simple coalition formation game. *Social Choice and Welfare*, 18(1):135–153, 2001. doi:10.1007/s003550000067.
- 4 Péter Biró, Gergely Csáji, and Ildikó Schlotter. Stable hypergraph matching in unimodular hypergraphs. arXiv:2502.08827 [cs.GT], 2025. doi:10.48550/arXiv.2502.08827.
- 5 Péter Biró and Tamás Fleiner. Fractional solutions for capacitated NTU-games, with applications to stable matchings. *Discrete Optimization*, 22:241–254, 2016. doi:10.1016/j.disopt.2015.02.002.
- 6 Alain Bretto. *Hypergraph Theory: An Introduction*. Mathematical Engineering. Springer Cham, 2013.
- 7 Karthekeyan Chandrasekaran, Yuri Faenza, Chengyue He, and Jay Sethuraman. Scarf’s algorithm on arborescence hypergraphs. arXiv:2412.03397 [cs.DM], 2024. doi:10.48550/arXiv.2412.03397.
- 8 Gergely Csáji. On the complexity of stable hypergraph matching, stable multicommodity flow and related problems. *Theoretical Computer Science*, 931:1–16, 2022. doi:10.1016/j.tcs.2022.07.025.
- 9 Yuri Faenza, Chengyue He, and Jay Sethuraman. Scarf’s algorithm and stable marriages. arXiv:2303.00791 [math.CO], 2023. doi:10.48550/arXiv.2303.00791.
- 10 Rita Fleiner, András Ferkai, and Péter Biró. College admission problem for university dual education. In *Proceedings of the IEEE 17th World Symposium on Applied Machine Intelligence and Informatics (SAMI 2019)*, pages 31–36. IEEE, 2019. doi:10.1109/SAMI.2019.8782783.
- 11 Tamás Fleiner. On the stable b -matching polytope. *Mathematical Social Sciences*, 46(2):149–158, 2003. doi:10.1016/S0165-4896(03)00074-X.
- 12 David Gale and Lloyd S. Shapley. College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1):9–15, 1962. doi:10.2307/2312726.
- 13 Guillaume Haeringer. *Market Design: Auctions and Matching*. MIT Press, 2018.
- 14 Shiva Kintali, Laura J. Poplawski, Rajmohan Rajaraman, Ravi Sundaram, and Shang-Hua Teng. Reducibility among fractional stability problems. *SIAM Journal on Computing*, 42(6):2063–2113, 2013. doi:10.1137/120874655.
- 15 László Lovász. Normal hypergraphs and the perfect graph conjecture. *Discrete Mathematics*, 2(3):253–267, 1972. doi:10.1016/0012-365X(72)90006-4.
- 16 David F. Manlove. *Algorithmics of Matching Under Preferences*, volume 2 of *Series on Theoretical Computer Science*. World Scientific, 2013. doi:10.1142/8591.
- 17 Alvin E. Roth. The evolution of the labor market for medical interns and residents: A case study in game theory. *Journal of Political Economy*, 92(6):991–1016, 1984. doi:10.1086/261272.
- 18 Alvin E. Roth and Marilda A. O. Sotomayor. *Two-sided Matching: a Study in Game-theoretic Modeling and Analysis*. Econometric Society Monographs, Cambridge, 1990.
- 19 Herbert E. Scarf. The core of an N person game. *Econometrica*, 35(1):50–69, 1967. doi:10.2307/1909383.
- 20 Alexander Schrijver. *Combinatorial Optimization*. Springer-Verlag Berlin Heidelberg, 2003.
- 21 Gerhard J. Woeginger. Core stability in hedonic coalition formation. In *Proceedings of the 39th International Conference on Current Trends in Theory and Practice of Computer Science (SOFSEM 2013)*, volume 7741 of *Lecture Notes in Computer Science*, pages 33–50. Springer, 2013. doi:10.1007/978-3-642-35843-2_4.